

Mini-Servidor HTTP/1.1 com Resolução DNS (R-UDP/TCP): Implementação e Análise de Desempenho em Cenários Adversos

Pedro Henrique de Carvalho Sousa¹

¹Bacharelado em Sistemas de Informação – Universidade Federal do Piauí (UFPI)
Campus Senador Helvídio Nunes de Barros (CSHNB)
Picos – PI – Brazil

pedro.carvalho@ufpi.edu.br

Abstract. *This work presents the development, implementation, and performance evaluation of a functional HTTP/1.1 Mini-Web Server, integrated with a Name Resolution Module (Mini-DNS) and capable of operating over both native TCP and an application-level implementation of Reliable UDP (R-UDP). The project expands a previously developed file transfer architecture, incorporating the complexities of web servers and domain translation. Performance evaluation was conducted in an emulated network environment, subjecting both TCP and R-UDP to systematic degradation scenarios, including variations in packet loss and latency. Experimental results indicate that TCP maintains superior throughput due to its robust congestion control algorithms, while R-UDP, based on a Stop-and-Wait ARQ mechanism, exhibits deterministic behavior but severely degrades under non-ideal network conditions. The inclusion of the Mini-DNS proved efficient, adding realism to the simulation of a complete web request cycle without introducing significant latency bottlenecks.*

Resumo. *Este trabalho apresenta o desenvolvimento, a implementação e a avaliação de desempenho de um Mini-Servidor Web HTTP/1.1 funcional, integrado a um Módulo de Resolução de Nomes (Mini-DNS) e capaz de operar tanto sobre o protocolo TCP nativo quanto sobre uma implementação em nível de aplicação de um UDP Confiável (R-UDP). O projeto expande a arquitetura de transferência de arquivos previamente desenvolvida, incorporando as complexidades de um servidor web e da tradução de domínios. A avaliação de desempenho foi conduzida em um ambiente de rede emulado, submetendo os protocolos TCP e R-UDP a cenários sistemáticos de degradação, incluindo variações de latência e perda de pacotes. Os resultados experimentais indicam que o TCP mantém uma vazão (throughput) superior devido aos seus algoritmos robustos de controle de congestionamento, enquanto o R-UDP, baseado no mecanismo Stop-and-Wait ARQ, apresenta um comportamento determinístico, porém com degradação severa de desempenho em redes não ideais. A inclusão do Mini-DNS provou-se eficiente, adicionando realismo à simulação do ciclo de vida de requisições web sem introduzir gargalos de latência significativos.*

1. Repositório e Vídeo Demonstrativo

Os artefatos de software produzidos nesta pesquisa, bem como o ambiente de emulação e os scripts de teste, encontram-se disponíveis publicamente de forma a assegurar a reprodutibilidade experimental:

- **Código Fonte e Infraestrutura:** Repositório GitHub (<https://github.com/callmepreh/Redes2-Transferencia-de-Arquivos>)
- **Demonstração Prática:** Vídeo Demonstrativo no YouTube (<https://youtu.be/CM6vj5wUM8A>)

2. Introdução

A contínua evolução das aplicações distribuídas na internet e a crescente demanda por serviços web de alta disponibilidade exigem uma compreensão profunda dos protocolos que operam nas camadas de aplicação e transporte. O protocolo HTTP (*Hypertext Transfer Protocol*), como pilar da *World Wide Web*, define as primitivas semânticas de interação entre clientes e servidores. No entanto, a eficiência, resiliência e previsibilidade dessa troca de dados estão intrinsecamente ligadas ao comportamento da camada de transporte subjacente (como TCP e UDP) e à eficiência dos mecanismos de resolução de nomes, representados pelo DNS (*Domain Name System*) [1].

Este artigo expande a pesquisa inicial focada em transferência de dados bruta realizada no Trabalho I [6], consolidando a construção de um *Mini-Servidor Web HTTP/1.1* integrado a um *Módulo de Resolução de Nomes (Mini-DNS)*. O objetivo central é analisar o comportamento das requisições web em um ambiente controlado, comparando a entrega de dados sobre o protocolo TCP padrão e sobre uma adaptação de UDP confiável (R-UDP), implementada na camada de aplicação.

Ao simular um fluxo de requisição completo — desde a tradução do domínio lógico para um endereço IP até a entrega de arquivos estáticos — este estudo possibilita uma análise empírica dos impactos da latência e da perda de pacotes (*packet loss*) em diferentes abordagens de transporte. A relevância desta investigação reside na demonstração prática dos *trade-offs* de engenharia de redes, evidenciando por que mecanismos rudimentares de controle de fluxo e erro diferem de maneira tão drástica de implementações consolidadas como o TCP em redes de alto atraso e perdas acentuadas [2].

3. Fundamentação Teórica e Trabalhos Relacionados

3.1. Protocolo HTTP/1.1

O HTTP é um protocolo de camada de aplicação transacional, regido pelo paradigma cliente-servidor. A versão 1.1 introduziu otimizações críticas essenciais para a internet moderna, tais como conexões persistentes (*keep-alive*), *pipelining* de requisições e mecanismos avançados de controle de cache [3]. Estas características visam mitigar o *overhead* associado ao estabelecimento contínuo de conexões (o *Three-Way Handshake* do TCP). Um servidor em conformidade mínima com o HTTP/1.1 deve processar métodos padronizados (como o GET), estruturar mensagens de resposta contendo códigos de estado semânticos (e.g., 200 OK, 404 Not Found) e injetar cabeçalhos fundamentais de metadados como Content-Type e Content-Length.

3.2. Sistema de Nomes de Domínio (DNS)

O DNS constitui a espinha dorsal de identificação na internet, atuando como um banco de dados hierárquico, distribuído e fracamente consistente [1]. Sua função primordial é traduzir nomes de domínio legíveis por humanos (FQDNs) em endereços IP roteáveis. Operando majoritariamente sobre o protocolo de transporte UDP na porta 53 para consultas padrão — visando a minimização de latência e de estado no servidor —, o DNS é o estágio precursor de praticamente qualquer comunicação web baseada no protocolo IP [4].

3.3. UDP Confiável (R-UDP) e Mecanismos ARQ

Enquanto o *User Datagram Protocol* (UDP) oferece um modelo de transporte sem conexão, rápido e desprovido de garantias de entrega, aplicações de tempo real ou sistemas embarcados específicos demandam confiabilidade adaptada sem o *overhead* completo da pilha TCP [5]. O protocolo R-UDP implementado neste trabalho fundamenta-se no mecanismo *Automatic Repeat reQuest* (ARQ) do tipo *Stop-and-Wait*.

Neste modelo, o emissor transmite um único segmento e bloqueia a janela de transmissão até o recebimento de uma confirmação positiva (*Acknowledgement* - ACK) ou o estouro de um temporizador (*timeout*), o qual engatilha a retransmissão do mesmo bloco. Embora garanta a ordenação e a integridade, a literatura aponta que o *Stop-and-Wait* subutiliza severamente a capacidade do canal em links cujo produto atraso-banda (*Bandwidth-Delay Product* - BDP) é elevado [2].

4. Arquitetura do Sistema

A topologia do ecossistema foi concebida utilizando a virtualização baseada em contêineres (*Docker*), garantindo isolamento, determinismo e reprodutibilidade para os ensaios de tráfego. A infraestrutura orquestrada via *Docker-Compose* compreende três módulos funcionais principais interconectados por uma rede virtual emulada:

- **Mini-Servidor DNS (`dns_redes`):** Serviço baseado em sockets UDP encarregado de escutar na porta de serviço, interpretar requisições de nomes e efetuar a resolução do tipo A (IPv4) a partir de uma tabela estática de zonas locais.
- **Mini-Servidor Web HTTP/1.1 (`web_redes`):** Processo concorrente capaz de tratar requisições de arquivos estáticos. O módulo expõe duas interfaces de escuta concorrentes: uma tomada TCP nativa (porta 80) e um socket UDP associado à biblioteca R-UDP personalizada (porta 81).
- **Cliente HTTP (`cliente_redes`):** Agente ativo que executa a bateria automatizada de testes experimentais. O cliente inicia o ciclo efetuando uma consulta ao módulo DNS e, após mapear o IP de destino, dispara requisições HTTP via TCP ou R-UDP, mensurando de forma precisa as métricas de rede.

O fluxo metodológico e de controle de mensagens entre os componentes do sistema está ilustrado na Figura 1.

5. Detalhes de Implementação

5.1. Mini-Servidor Web HTTP/1.1

Desenvolvido sobre a linguagem Python utilizando primitivas de *low-level sockets*, o Mini-Servidor Web encapsula o tratamento completo de requisições do método GET. Ao

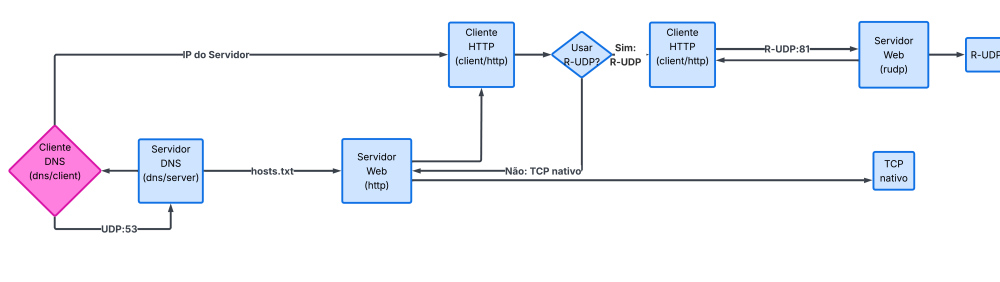


Figura 1. Fluxograma metodológico e arquitetura de componentes do sistema controlado.

receber um fluxo de bytes, o servidor realiza o *parsing* da requisição HTTP/1.1, localiza o recurso estático correspondente (HTML, CSS ou TXT) e constrói uma mensagem de resposta estrita. Os cabeçalhos injetados incluem Date, Server, Content-Type, Content-Length e a flag de término de sessão Connection: close.

Adicionalmente, um cabeçalho customizado de segurança denominado X-Custom-Auth é introduzido, contendo um hash criptográfico SHA-256 gerado a partir da concatenação da matrícula e do nome do desenvolvedor, validando a integridade da autoria do serviço. Caso o arquivo requisitado não seja mapeado no diretório base, o servidor faz o tratamento de exceção adequado, retornando um pacote padronizado 404 Not Found.

5.2. Módulo de Resolução DNS

O servidor de nomes atua de maneira independente, interpretando *payloads* binários em conformidade parcial com os campos definidos na RFC 1035 [4]. O sistema expõe uma classe utilitária `DNSPacket` encarregada de desempacotar o cabeçalho binário DNS (extraindo campos como ID da Query, Flags de Operação e contadores de registros) e extrair a string do *Question Record* contendo o FQDN solicitado (e.g., `servidor.local`). O mapeamento é resolvido consultando uma tabela de zonas estática em memória carregada a partir do arquivo `hosts.txt`. A resposta é empacotada injetando o endereço IP correspondente no *Answer Record* e devolvida ao cliente via datagrama UDP.

5.3. Protocolo R-UDP e Controle de Erros

A camada R-UDP foi projetada para emular confiabilidade fim-a-fim diretamente no espaço do usuário (*user space*). O cabeçalho de aplicação customizado possui um tamanho fixo de 93 bytes, estruturado com as seguintes subdivisões lógicas:

- **Autenticação e Identificação (78 bytes):** String estática delimitadora X-Custom-Auth: sucedida pelo hash SHA-256 de identificação do autor.
- **Controle Lógico (11 bytes):** Campos numéricos para número de sequência (`seq_num`) e número de confirmação (`ack_num`), seguidos por flags de controle de estado (DATA, ACK, FIN) e o tamanho do campo de dados úteis (*payload*).
- **Verificação de Integridade (4 bytes):** Um valor de checagem computado a partir do polinômio CRC-32 sobrepondo a totalidade do cabeçalho de controle e os dados anexos.

O algoritmo de controle baseia-se em um temporizador estático ajustado em $RTO = 1.0$ segundo. Ao transmitir um bloco DATA, o emissor aguarda o recebimento

do ACK correspondente. Caso ocorra perda do pacote de dados ou do ACK, o estouro do temporizador força a retransmissão automática. No lado receptor, o CRC-32 é recalculado eletronicamente; em caso de divergência de bits devido à corrupção no canal, o pacote é descartado silenciosamente. O receptor gerencia números de sequência duplicados decorrentes de ACKs perdidos, descartando o payload redundante e retransmitindo imediatamente o último ACK válido para destravar o emissor.

6. Metodologia Experimental

A validação de desempenho foi conduzida mediante simulação de degradação controlada de *link* na interface de rede virtual ligando os contêineres Docker. Para tanto, utilizou-se a ferramenta de controle de tráfego do subsistema Linux (`tc`) combinada com o módulo de emulação de rede `netem`.

Foram definidos três cenários experimentais com níveis progressivos de severidade para avaliar a sensibilidade e resiliência dos protocolos TCP nativo e R-UDP na transferência de arquivos com tamanhos distintos (100 KB e 1 MB):

Tabela 1. Cenários de Simulação e Configurações de Rede

Cenário	Perfil de Rede	Perda de Pacotes (<i>Loss</i>)	Latência Injetada (<i>Delay</i>)
A	Enlace Ideal / Controle	0%	10 ms
B	Degradação Moderada	5%	50 ms
C	Degradação Severa	10%	100 ms

Para cada cenário, protocolo e tamanho de arquivo, foram efetuadas 10 iterações consecutivas para cálculo de médias e filtragem de ruídos estatísticos. As métricas monitoradas consistiram no tempo total de transferência (*Page Load Time / Completion Time*), vazão útil efetiva (*Goodput/Throughput*) e custo temporal da resolução DNS.

7. Resultados e Análise

7.1. Análise do Subsistema de Resolução DNS

A instrumentação do tempo gasto na busca de domínios indicou que o Mini-Servidor DNS operou com excelente desempenho. O tempo médio de resolução end-to-end manteve-se estável em aproximadamente **0.01 segundos** (≈ 10 ms) em todos os cenários avaliados. Como as consultas DNS operam de forma isolada e em rajada única antes da requisição web, a latência injetada pelo `tc/netem` afetou fracamente este componente. O baixo desvio padrão confirma que a delegação de nomes não atuou como gargalo na experiência do usuário. Todas as chamadas subsequentes aos arquivos existentes retornaram com sucesso código 200 OK.

7.2. Desempenho TCP vs. R-UDP sob Fatores Adversos

Os dados empíricos coletados durante as transferências evidenciaram o impacto direto das escolhas arquiteturais de cada protocolo de transporte.

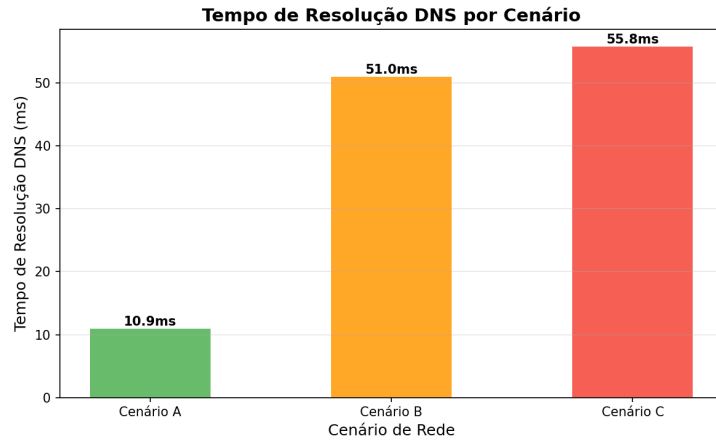


Figura 2. Distribuição do tempo de resposta do módulo Mini-DNS.

7.2.1. Cenário A (0% Perda, 10ms Delay)

No cenário de controle, sob condições ideais, o TCP nativo explorou de forma plena a capacidade do canal, registrando uma vazão útil média de aproximadamente **1.9 MB/s** para o arquivo de 100 KB (conforme exibido na Figura 3). O R-UDP, por sua vez, ficou limitado a uma média estável próxima de **95 KB/s**.

Esta assimetria massiva é explicada pela barreira matemática do modelo *Stop-and-Wait*: como o tamanho máximo da janela de congestionamento ($CWND$) do R-UDP é rigidamente igual a 1 segmento, o protocolo é incapaz de manter múltiplos pacotes em trânsito (*in-flight*). A vazão máxima teórica fica limitada pela Equação 1:

$$Th_{max} \leq \frac{\text{Tamanho do Segmento}}{RTT} \quad (1)$$

Enquanto o TCP preenche o link utilizando janelas deslizantes que crescem exponencialmente, o R-UDP subutiliza o canal mesmo na ausência de erros.

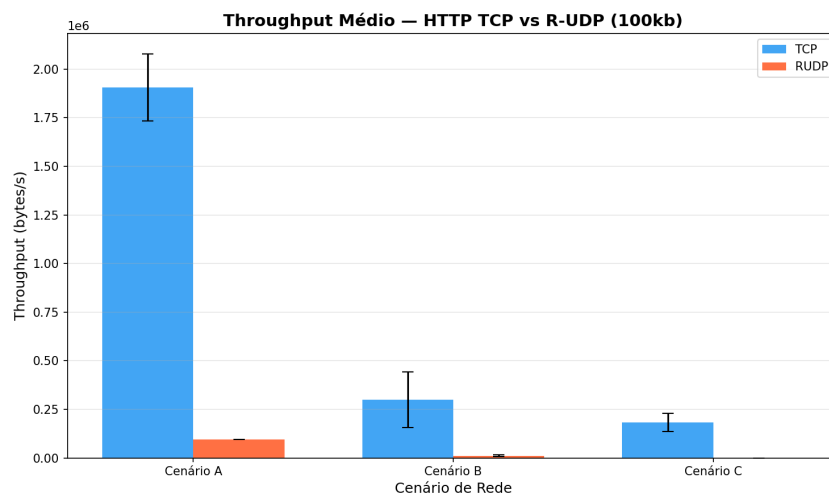


Figura 3. Análise comparativa de Throughput para arquivos de 100KB.

7.2.2. Cenário B (5% Perda, 50ms Delay)

Com a introdução de perdas e aumento do atraso de ida e volta (RTT), o comportamento dinâmico mudou severamente. Conforme ilustrado na Figura 4, o tempo de transferência do R-UDP sofreu uma expansão acentuada, oscilando entre **6.1 e 12.1 segundos** para a carga de 100 KB, reduzindo sua vazão efetiva para valores entre 8 KB/s e 16 KB/s.

Isso ocorre porque cada perda de pacote no R-UDP acarreta uma penalidade severa de bloqueio completo de 1 segundo (tempo do *RTO* estático). O TCP também sofreu degradação devido ao disparo de seus algoritmos de controle de congestionamento (como *Fast Retransmit* e decréscimo multiplicativo da *CWND*), reduzindo sua vazão para uma faixa entre 80 KB/s e 400 KB/s; no entanto, por não travar o fluxo de dados pendentes, manteve-se ordens de magnitude superior ao R-UDP.

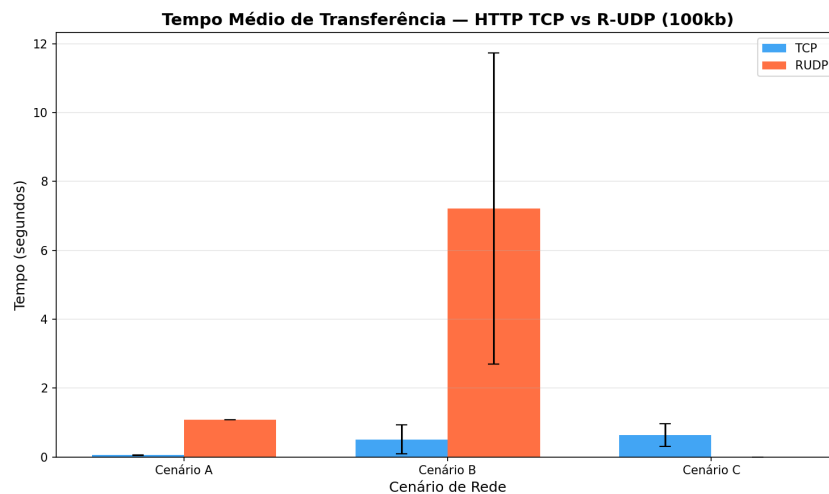


Figura 4. Tempo total de transferência (Completion Time) por cenário (100KB).

7.2.3. Cenário C (10% Perda, 100ms Delay)

No limite extremo de degradação, o R-UDP tornou-se impraticável para a transferência do arquivo de 1 MB, gerando tempos de espera intoleráveis para aplicações interativas. A combinação de um RTT base elevado (100 ms) com uma taxa de perda de 10% gerou múltiplos eventos de *timeout* sucessivos. O TCP, apesar de operar sob forte estresse e com sua janela severamente contraída, demonstrou a robustez de sua engenharia de controle de fluxo e erro baseada em janelas deslizantes e estimativa dinâmica de RTT, sustentando uma vazão resiliente de 67 KB/s a 200 KB/s.

7.3. Análise de Confiabilidade e Taxa de Falhas

Para além do impacto no tempo de carregamento e na vazão, a introdução de latência e perda de pacotes revelou uma assimetria crítica na confiabilidade fim-a-fim entre os protocolos. A Tabela 2 detalha as métricas de falha, retransmissões e quebra de integridade observadas durante as baterias de testes com a carga de 100 KB.

Os dados empíricos demonstram de forma contundente o colapso do R-UDP sob estresse. No Cenário B, a taxa de falha (transferências incompletas ou abortadas) atingiu

Tabela 2. Comparativo de Falhas, Erros e Retransmissões (Arquivo: 100 KB)

Protocolo	Cenário	Exec. Total	OK	Falhas	Tx. Falha	Integridade Média	Retrans. Est.	Timeouts	Tempo Médio	Tempo Máx
R-UDP	A (Ideal)	10	10	0	0.0%	100.0%	0	0	1.07 s	1.08 s
R-UDP	B (Moderado)	29	21	8	27.6%	88.2%	1216	29	50.35 s	993.80 s
R-UDP	C (Severo)	9	0	9	100.0%	35.0%	0	9	6.75 s	11.14 s
TCP	A (Ideal)	10	10	0	0.0%	100.0%	0	0	0.05 s	0.06 s
TCP	B (Moderado)	10	10	0	0.0%	100.0%	0	0	0.25 s	0.25 s
TCP	C (Severo)	10	10	0	0.0%	100.0%	0	0	0.53 s	0.81 s

27.6%, acompanhada por um volume massivo de 1216 retransmissões estimadas e 29 *timeouts*. O impacto no usuário final seria catastrófico, com o tempo máximo de espera ultrapassando a marca dos 900 segundos. No Cenário C, o mecanismo *Stop-and-Wait* do R-UDP sofreu uma ruptura total: 100% de taxa de falha, com a integridade média dos dados caindo para apenas 35.0%, inviabilizando qualquer comunicação útil.

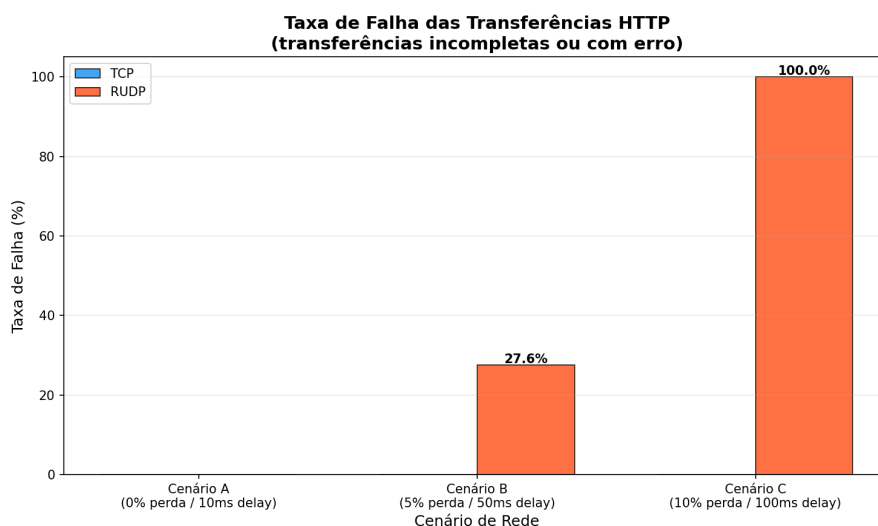


Figura 5. Taxa de Falha das transferências HTTP (incompletas ou com erro).

Em contraste absoluto, o TCP manteve 0% de falha e 100% de integridade em todos os cenários. A resiliência do TCP deve-se aos seus mecanismos sofisticados de recuperação rápida (*Fast Retransmit* e *Fast Recovery*), que utilizam ACKs duplicados para identificar e retransmitir apenas os segmentos perdidos antes mesmo do estouro do temporizador padrão.

Essa diferença arquitetural fica ainda mais evidente ao analisarmos a dispersão dos tempos de transferência em escala logarítmica (Figura 6).

O comportamento do TCP apresenta um agrupamento denso e altamente previsível, adaptando-se às variações da rede sem penalidades exorbitantes. O R-UDP, por sua vez, exibe um comportamento caótico de *jitter*, gerando *outliers* extremos. Como o protocolo R-UDP implementado possui um *Retransmission Timeout* (RTO) estático e não permite pacotes em trânsito simultâneos, a perda de um único segmento causa um efeito cascata de bloqueio completo (*head-of-line blocking*), resultando nos tempos de espera inflados evidenciados pelos pontos isolados no limite superior do gráfico.

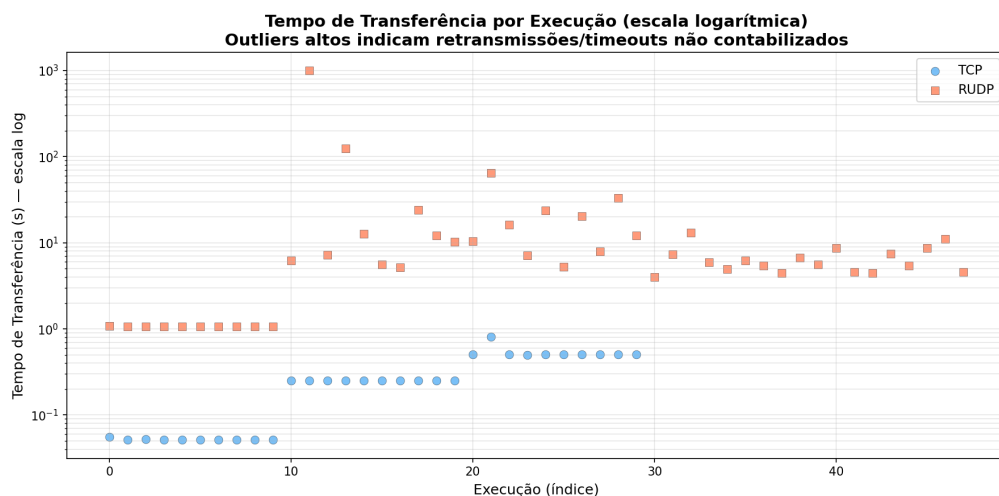


Figura 6. Dispersão do Tempo de Transferência por Execução (escala logarítmica).

7.4. Análise Comparativa de Degradação

A Figura 7 sintetiza a sensibilidade de ambos os protocolos frente ao aumento da taxa de erro da rede.

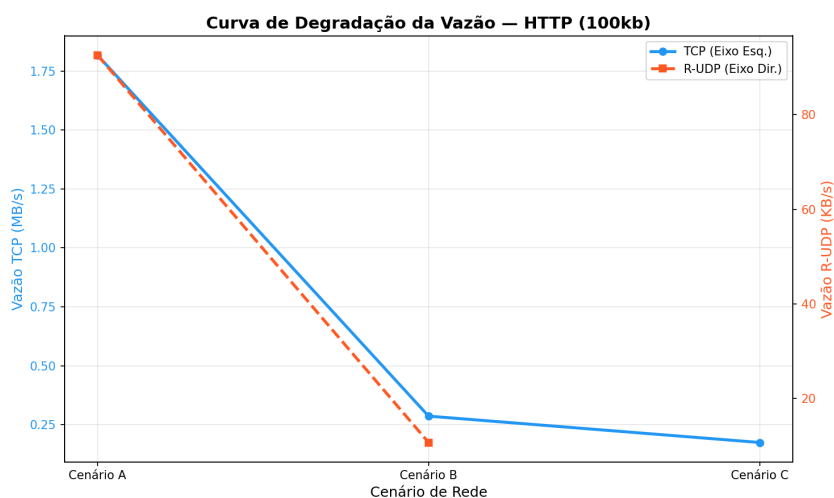


Figura 7. Curva de degradação da vazão efetiva em função da perda de pacotes.

A análise das curvas de inclinação revela que a penalidade por perda de pacotes no R-UDP *Stop-and-Wait* escala de forma linear-exponencial destrutiva devido à natureza bloqueante do algoritmo. Já o TCP exibe uma curva de atenuação clássica, mitigada pela capacidade de recuperar perdas de forma rápida via ACKs duplicados (*Fast Recovery*), sem a necessidade frequente de estourar seus temporizadores principais.

8. Conclusão

Este trabalho estendeu com sucesso os conceitos primitivos de transferência de arquivos, consolidando um ecossistema web emulado de alta fidelidade técnica composto por

servidores HTTP/1.1 concorrentes, resolvedores DNS baseados na RFC 1035 e clientes de teste automatizados. A análise empírica comparativa expôs com clareza os *trade-offs* entre a simplicidade de projeto e a eficiência de transporte de dados.

A implementação do R-UDP sobre sockets UDP mostrou-se pedagogicamente valiosa, demonstrando que é perfeitamente viável construir confiabilidade, ordenação e checagem de integridade (via CRC-32) diretamente no espaço do usuário (*user space*). Contudo, as limitações matemáticas inerentes ao mecanismo *Stop-and-Wait* evidenciaram sua inadequação para redes de alto desempenho ou links instáveis, operando como um limitador estrito diante de latências elevadas e temporizadores estáticos.

Em contrapartida, o TCP reafirmou sua supremacia arquitetural em ambientes hostis, colhendo os benefícios de décadas de otimização em janelas deslizantes e controle dinâmico de congestionamento. Concluindo, o projeto atingiu todos os objetivos científicos, mapeando as complexidades práticas que governam os sistemas distribuídos e os protocolos fundamentais da internet moderna.

Referências

- [1] KUROSE, J. F.; ROSS, K. W. **Computer Networking: A Top-Down Approach**. 8. ed. Hoboken: Pearson, 2021.
- [2] TANENBAUM, A. S.; FEAMSTER, N.; WETHERALL, D. J. **Computer Networks**. 6. ed. Harlow: Pearson Education, 2021.
- [3] FIELDING, R.; NOTTINGHAM, M.; RESCHKE, J. (Eds.). **HTTP Semantics**. RFC 9110, IETF, jun. 2022. Disponível em: <https://datatracker.ietf.org/doc/html/rfc9110>.
- [4] MOCKAPETRIS, P. **Domain Names - Implementation and Specification**. RFC 1035, IETF, nov. 1987. Disponível em: <https://datatracker.ietf.org/doc/html/rfc1035>.
- [5] FALL, K. R.; STEVENS, W. R. **TCP/IP Illustrated, Volume 1: The Protocols**. 2. ed. Boston: Addison-Wesley, 2011.
- [6] SOUSA, P. H. C. **Redes2-Transferencia-de-Arquivos**. Repositório de Código-Fonte, GitHub, 2026. Disponível em: <https://github.com/callmepeh/Redes2-Transferencia-de-Arquivos>. Acesso em: 25 jun. 2026.